

**AVALIAÇÃO TÉCNICA DE UMA ARQUITETURA WEB BASEADA EM
BAAS/SERVERLESS PARA INTERMEDIACÃO DE SERVIÇOS:
UM ESTUDO DE CASO DA PLATAFORMA HUBSERVI**

Pedro Conrado Fernandes Vieira

Graduando em Engenharia de Software – Uni-FACEF

Email: opeconrado@gmail.com

Richardy Gabriel Rodrigues da Costa

Graduando em Engenharia de Software – Uni-FACEF

Email: richardyrodrigues519@gmail.com

Prof. Daniel Facciolo Pires

Docente do Departamento de Computação – Uni-FACEF

Email: daniel@facef.br

Resumo

Arquiteturas web fundamentadas em *Backend as a Service* (BaaS) e em computação *serverless* têm sido amplamente adotadas no desenvolvimento de aplicações, por reduzirem o esforço de implementação e operação da infraestrutura de *backend*. Entretanto, equipes de desenvolvimento enfrentam dificuldade em validar tecnicamente se essas arquiteturas atendem aos atributos de qualidade esperados, como segurança, desempenho, testabilidade e manutenibilidade. Este trabalho tem como objetivo avaliar tecnicamente a arquitetura de software da plataforma Hubservi — uma aplicação web de intermediação de serviços construída como *Single Page Application* (SPA) em React e TypeScript, apoiada pelo BaaS Supabase sobre PostgreSQL —, utilizando métricas e testes de Engenharia de Software relacionados a segurança, desempenho, testabilidade, manutenibilidade e confiabilidade, à luz da norma ISO/IEC 25010. A pesquisa caracteriza-se como aplicada, de abordagem mista, com objetivos exploratórios e descritivos, conduzida por meio de estudo de caso combinado a experimento técnico. Como contribuição, propõe-se um procedimento de avaliação arquitetural que articula a norma ISO/IEC 25010, o método ATAM (*Architecture Tradeoff Analysis Method*) e um conjunto de ferramentas de teste automatizado, análise estática, desempenho e segurança. O procedimento foi executado sobre ambiente reprodutível e emitiu vereditos por atributo e por camada: três defeitos reais de autorização e de integridade foram detectados, corrigidos e remediados; a confiabilidade atende aos critérios definidos; o *backend* atende ao critério de desempenho, com latência de cauda (p97,5) de 253 ms e 0% de erro sob carga, ao passo que o carregamento inicial do *frontend* não o atinge, com LCP de 3,17 s após otimização, contra a meta de 2,5 s; e a manutenibilidade apresenta estrutura sólida, sem dependências circulares, com higiene de código abaixo do ideal. A capacidade de localizar deficiências específicas — e não de atestar qualidade uniforme — evidencia o valor do procedimento proposto.

Palavras-chave: Arquitetura de software. Avaliação arquitetural. ISO/IEC 25010. Backend as a Service. Serverless.

Abstract

Web architectures based on *Backend as a Service* (BaaS) and *serverless* computing have been widely adopted in application development, as they reduce the effort of implementing and operating backend infrastructure. However, development teams struggle to technically validate whether such architectures meet the expected quality attributes, such as security, performance, testability, and maintainability. This work aims to technically evaluate the software architecture of the Hubservi platform — a web application for service intermediation built as a *Single Page Application* (SPA) using React and TypeScript, supported by the Supabase BaaS over PostgreSQL —, using Software Engineering metrics and tests related to security, performance, testability, maintainability, and reliability, in light of the ISO/IEC 25010 standard. The research is characterized as applied, with a mixed approach and exploratory and descriptive objectives, conducted through a case study combined with a technical experiment. As a contribution, it proposes an architectural evaluation procedure that articulates the ISO/IEC 25010 standard, the ATAM (*Architecture Tradeoff Analysis Method*), and a set of tools for automated testing, static analysis, performance, and security. The procedure was executed on a reproducible environment and issued verdicts per attribute and per layer: three real authorization and integrity defects were detected, fixed, and re-measured; reliability meets the defined criteria; the backend meets the performance criterion, with a tail latency (p97.5) of 253 ms and a 0% error rate under load, whereas the initial frontend load does not, with an LCP of 3.17 s after optimization against a 2.5 s target; and maintainability shows a sound structure, free of circular dependencies, with below-ideal code hygiene. The ability to pinpoint specific deficiencies — rather than to certify uniform quality — demonstrates the value of the proposed procedure.

Keywords: Software architecture. Architectural evaluation. ISO/IEC 25010. Backend as a Service. Serverless.

1 INTRODUÇÃO

1.1 Contexto

A intermediação de serviços — atividade que conecta pessoas que demandam um serviço a profissionais capazes de executá-lo — tem migrado progressivamente para plataformas digitais. Aplicações web nesse domínio precisam suportar cadastro e descoberta de serviços, solicitação e gerenciamento de atendimentos e avaliação reputacional, com requisitos não triviais de segurança, desempenho e confiabilidade, uma vez que manipulam dados pessoais e transações entre partes que, em geral, não se conhecem previamente.

Em paralelo, o modelo de desenvolvimento de aplicações web tem sido reconfigurado pela popularização de arquiteturas baseadas em *Backend as a Service* (BaaS) e em computação *serverless*. Nesses modelos, responsabilidades tradicionalmente implementadas em servidores próprios — autenticação, persistência, autorização e regras de acesso — são delegadas a serviços gerenciados por terceiros, acessados diretamente pelo cliente por meio de APIs e bibliotecas. Plataformas como o Supabase materializam esse paradigma ao expor, sobre um banco PostgreSQL, autenticação integrada e autorização declarativa via *Row Level Security* (RLS), eliminando boa parte da camada de servidor de aplicação convencional.

A plataforma Hubservi, objeto deste estudo, é uma aplicação web de intermediação de serviços construída como *Single Page Application* (SPA) em React e TypeScript, apoiada pelo Supabase. Trata-se, portanto, de um caso representativo da arquitetura **SPA + BaaS + Serverless**, na qual a lógica de negócio se distribui entre o cliente (validações, fluxos de interface e orquestração de chamadas) e o banco de dados (regras declarativas de autorização, *triggers* e *views*).

1.2 Problema de pesquisa

A adoção de arquiteturas BaaS/Serverless reduz o esforço de implementação e operação de infraestrutura, mas desloca atributos de qualidade críticos — em especial a segurança e a confiabilidade — para configurações declarativas (políticas de RLS, *triggers*, restrições de integridade) e para a fronteira cliente–serviço gerenciado. Esse deslocamento dificulta a verificação de que a arquitetura efetivamente atende aos atributos de qualidade esperados: a ausência de uma camada de servidor de aplicação própria altera o que deve ser testado, onde residem os pontos de falha e como o desempenho e a manutenibilidade devem ser medidos.

Formaliza-se, assim, o problema de pesquisa:

Equipes de desenvolvimento de aplicações web têm dificuldade em validar tecnicamente se uma arquitetura baseada em *Backend as a Service* (BaaS) e *Serverless* atende aos atributos de qualidade esperados, como segurança, desempenho, testabilidade e manutenibilidade.

1.3 Questão de pesquisa

Decorre do problema a seguinte questão de pesquisa:

Como avaliar tecnicamente uma arquitetura web baseada em BaaS/Serverless por meio de métricas e testes de Engenharia de Software, considerando atributos de qualidade em uma plataforma de intermediação de serviços?

1.4 Objetivos

1.4.1 Objetivo geral

Avaliar tecnicamente a arquitetura de software da plataforma Hubservi, baseada em React, TypeScript, Supabase e PostgreSQL, utilizando métricas e testes relacionados à segurança, ao desempenho, à testabilidade e à manutenibilidade.

1.4.2 Objetivos específicos

1. Identificar os requisitos arquiteturais e os atributos de qualidade relevantes para a plataforma.
2. Modelar a arquitetura da plataforma.
3. Documentar os componentes, fluxos, persistência e regras de negócio.
4. Definir cenários de avaliação arquitetural.
5. Executar testes automatizados.
6. Executar análise estática de código.
7. Executar testes de segurança.
8. Executar testes de desempenho.
9. Analisar os resultados obtidos.

Nota de escopo. Os objetivos específicos 1 a 4 (delimitação, modelagem, documentação e definição de cenários) e os objetivos 5 a 9 (execução de testes automatizados, análise estática, testes de segurança e de desempenho, e análise) foram executados; os resultados são reportados na Seção 7 e derivam do registro reproduzível em *docs/tcc/medicoes/*.

1.5 Delimitação do escopo

Objeto de pesquisa e instrumento. O objeto deste trabalho é a **avaliação técnica** de uma arquitetura web baseada em BaaS/Serverless; a plataforma Hubservi é o **instrumento** por meio do qual essa avaliação se torna possível. A distinção é necessária porque o trabalho

envolve dois produtos de naturezas distintas: um sistema de software, desenvolvido integralmente pela equipe, e um procedimento de avaliação, aplicado sobre esse sistema. Apenas o segundo constitui a contribuição científica pretendida. Conforme exposto na Seção 1.6, a lacuna identificada é de ordem metodológica — não é evidente *como* aplicar métricas e testes de Engenharia de Software para validar atributos de qualidade em um arranjo no qual autorização e integridade migram para o banco de dados. O que se oferece como contribuição, portanto, é um roteiro de avaliação reprodutível e transferível a outras aplicações que adotem o mesmo paradigma, e não a plataforma em si.

Justificativa do desenvolvimento do sistema. A construção do Hubservi não constitui objetivo do trabalho, mas **pré-requisito metodológico** dele. Avaliar políticas de RLS, *triggers*, restrições de integridade e a fronteira entre cliente e serviço gerenciado exige acesso irrestrito ao código-fonte, ao esquema do banco de dados, às configurações de autorização e ao ambiente de execução — condições inviáveis de obter sobre uma plataforma de terceiros, cujo código e cuja base de dados não são acessíveis ao pesquisador. O desenvolvimento próprio é o que assegura a validade interna do experimento: permite controlar as variáveis do ambiente, reproduzir as medições sobre um estado conhecido e conduzir o ciclo de detecção e correção de defeitos sem restrições de acesso. Os quatro fluxos críticos implementados — autenticação, cadastro e busca de serviços, contratação (*booking*) e avaliação (*review*), descritos na Seção 4 — constituem a superfície sobre a qual os cenários de avaliação da Seção 5 são exercitados.

Delimitação negativa. Não integram o escopo deste trabalho: (i) a avaliação de **usabilidade e acessibilidade**, por estarem fora das quatro características da ISO/IEC 25010 (2011) selecionadas como relevantes para o problema de pesquisa — segurança, eficiência de desempenho, manutenibilidade e confiabilidade —, ainda que constem como requisitos não funcionais do produto; (ii) o **módulo de recomendação**, que permanece secundário e limitado à ordenação de resultados por popularidade e avaliação média, não sendo objeto de avaliação; (iii) arquiteturas de **microsserviços**, uma vez que o sistema avaliado adota o modelo SPA + BaaS + Serverless, sem camada de servidor de aplicação própria; e (iv) testes **fim a fim conduzidos por ferramenta dedicada de automação de navegador**, dado que, em uma arquitetura BaaS, a superfície de integração relevante é a própria API do serviço gerenciado — os fluxos ponta a ponta são, por essa razão, verificados por testes de integração executados contra a API, conforme a Seção 5.

1.6 Justificativa

A literatura de arquitetura de software dispõe de métodos consolidados de avaliação — notadamente o ATAM (*Architecture Tradeoff Analysis Method*), de Clements, Kazman e Klein (2002) — e de modelos de qualidade reconhecidos, como o estabelecido pela norma ISO/IEC 25010 (2011). Tais referenciais, contudo, foram concebidos predominantemente sob o pressuposto de arquiteturas com camada de servidor de aplicação explícita. A crescente adoção de arquiteturas BaaS/Serverless, nas quais responsabilidades de autorização e integridade migram para o banco de dados e para serviços gerenciados, cria uma lacuna prática: não é evidente *como* aplicar métricas e testes de Engenharia de Software para validar tecnicamente esses atributos de qualidade nesse novo arranjo.

Justifica-se, portanto, conduzir um estudo de caso que (i) modele e documente rigorosamente uma arquitetura BaaS/Serverless real, (ii) defina cenários e métricas de avaliação alinhados aos atributos de qualidade da ISO/IEC 25010, e (iii) organize um procedimento reprodutível de avaliação. A contribuição é tanto prática — para a equipe da plataforma Hubservi — quanto metodológica, ao oferecer um roteiro de avaliação técnica transferível a outras aplicações que adotem o mesmo paradigma.

1.7 Organização do artigo

O restante do artigo está organizado da seguinte forma. A Seção 2 apresenta o referencial teórico sobre arquitetura de software, avaliação arquitetural, qualidade de software e arquiteturas BaaS/Serverless. A Seção 3 descreve a metodologia adotada. A Seção 4 documenta a arquitetura da plataforma Hubservi. A Seção 5 detalha o planejamento experimental e o plano de métricas. A Seção 6 apresenta a avaliação arquitetural com base no ATAM. A Seção 7 consolida os resultados das medições executadas. A Seção 8 apresenta a conclusão, as limitações e os trabalhos futuros e, por fim, são listadas as referências.

1.8 Disponibilidade dos artefatos

Todo o material que sustenta este trabalho é público e versionado, em coerência com a própria tese do artigo: uma avaliação técnica só é verificável se o instrumento, o procedimento e as evidências puderem ser inspecionados por terceiros. O repositório (<https://github.com/Richardy-Rodrigues/hubservi>) reúne o código-fonte da plataforma, as *migrations* que definem o esquema e as políticas de autorização, as suítes de teste, o registro completo das medições — com ferramenta, versão, data, valor, critério e veredito — e as saídas brutas de cada execução.

Três documentos são publicados como **material suplementar**, referenciados ao longo do texto. O primeiro (VIEIRA; COSTA, 2026a) reúne os diagramas de modelagem — UML, BPMN e DER — e o dicionário de dados, numerados de forma autocontida (Figuras A.1 a A.10 e Tabelas A.1 a A.7). O segundo (VIEIRA; COSTA, 2026b) descreve como reproduzir cada medição reportada na Seção 7, classificando-as pelo que exigem do ambiente e explicitando quais **não** são reproduzíveis e por quê — informação que a Seção 5.5 trata como parte das ameaças à validade, e não como omissão. O terceiro (VIEIRA; COSTA, 2026c) reúne as capturas de tela de todas as execuções de ferramenta que sustentam a Seção 7, das quais as mais relevantes são reproduzidas no próprio corpo do artigo.

2 REFERENCIAL TEÓRICO

Esta seção fundamenta teoricamente o trabalho em quatro eixos: (i) arquitetura de software e atributos de qualidade; (ii) avaliação arquitetural e o método ATAM; (iii) qualidade de software segundo a ISO/IEC 25010; e (iv) o paradigma de arquiteturas BaaS/Serverless. Encerra com a contribuição da Engenharia de Software no que tange a testes e análise estática.

2.1 Arquitetura de software e atributos de qualidade

A arquitetura de software de um sistema é definida por Bass, Clements e Kazman (2012) como o conjunto de estruturas necessárias para raciocinar sobre o sistema, compreendendo elementos de software, as relações entre eles e as propriedades de ambos. Para os autores, a arquitetura é o artefato que viabiliza ou inibe os atributos de qualidade do sistema: decisões arquiteturais — e não primariamente decisões de implementação — determinam o grau em que requisitos como desempenho, segurança e modificabilidade serão satisfeitos.

Bass, Clements e Kazman (2012) distinguem requisitos funcionais, que expressam *o que* o sistema deve fazer, dos *requisitos de atributos de qualidade*, que expressam *quão bem* o sistema deve fazê-lo. Esses requisitos são expressos por meio de **cenários de atributos de qualidade**, estruturados em seis partes: fonte do estímulo, estímulo, artefato, ambiente, resposta e medida da resposta. Tal estrutura é central para este trabalho, pois fornece o formato pelo qual os atributos avaliados na plataforma Hubservi são operacionalizados como cenários verificáveis (Seções 5 e 6).

Os autores também introduzem o conceito de **táticas** — decisões de projeto que influenciam o controle de um atributo de qualidade — e de **ASR** (*Architecturally Significant Requirements*), os requisitos cuja satisfação depende de decisões arquiteturais. No contexto de

uma arquitetura BaaS/Serverless, táticas de segurança como *autenticar atores* e *autorizar atores* materializam-se em mecanismos declarativos (autenticação gerenciada e políticas de RLS), o que justifica avaliá-las de forma específica.

2.2 Avaliação arquitetural e o método ATAM

Clements, Kazman e Klein (2002), em *Evaluating Software Architectures*, argumentam que a arquitetura, por ser o primeiro artefato em que os atributos de qualidade do sistema se tornam analisáveis, pode e deve ser avaliada antes de a construção avançar, reduzindo o risco de retrabalho. Os autores propõem o **ATAM** (*Architecture Tradeoff Analysis Method*), método de avaliação baseado em cenários cujo objetivo não é fornecer notas precisas, mas identificar **riscos**, **pontos de sensibilidade** (*sensitivity points*) e **pontos de compromisso** (*tradeoff points*) decorrentes das decisões arquiteturais.

O ATAM organiza-se em torno de uma **árvore de utilidade** (*utility tree*), que decompõe a utilidade geral do sistema em atributos de qualidade, estes em refinamentos, e estes, por fim, em cenários priorizados segundo a importância para o negócio e o grau de risco arquitetural. Os principais conceitos do método, mobilizados na Seção 6, são:

- **Ponto de sensibilidade:** propriedade de um ou mais componentes da arquitetura que é crítica para se alcançar uma resposta de atributo de qualidade.
- **Ponto de compromisso:** propriedade que é ponto de sensibilidade para mais de um atributo, de modo que melhorá-la para um atributo pode degradar outro.
- **Risco e não risco:** decisões arquiteturais com consequências potencialmente negativas (ou explicitamente seguras) para os atributos de qualidade.

O método é especialmente adequado a este trabalho por ser orientado a cenários e por focalizar *tradeoffs*, dimensão central em arquiteturas BaaS/Serverless, nas quais a delegação de responsabilidades a serviços gerenciados implica compromissos explícitos entre simplicidade, controle, desempenho e segurança.

2.3 Qualidade de software e a norma ISO/IEC 25010

A norma ISO/IEC 25010 (2011), parte da família SQuaRE (*Systems and software Quality Requirements and Evaluation*), define um **modelo de qualidade do produto de software** composto por oito características, cada qual subdividida em subcaracterísticas. As características são: adequação funcional, eficiência de desempenho, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade.

Para o escopo deste trabalho, são mobilizadas as seguintes características e subcaracterísticas:

- **Segurança** (*security*): confidencialidade, integridade, não repúdio, responsabilização (*accountability*) e autenticidade. Avaliada por meio de autenticação, autorização e controle de acesso indevido.
- **Eficiência de desempenho** (*performance efficiency*): comportamento temporal, utilização de recursos e capacidade. Avaliada por tempo de resposta, tempo de carregamento e comportamento sob carga.
- **Manutenibilidade** (*maintainability*): modularidade, reusabilidade, analisabilidade, modificabilidade e **testabilidade**. Cabe registrar que, na ISO/IEC 25010 (2011), a testabilidade é uma subcaracterística da manutenibilidade; neste trabalho ela é tratada como dimensão avaliativa destacada, em razão de sua centralidade para a verificação dos demais atributos.

- **Confiabilidade** (*reliability*): maturidade, disponibilidade, tolerância a falhas e recuperabilidade. Avaliada pela consistência de operações e pelo comportamento em fluxos críticos.

A norma fornece, assim, o vocabulário e a taxonomia que estruturam o plano de métricas (Seção 5), garantindo rastreabilidade entre cada métrica coletada e a característica de qualidade que ela pretende evidenciar.

2.4 Arquiteturas BaaS e Serverless

O termo *serverless* designa um modelo de execução no qual a provisão, o escalonamento e a manutenção de servidores são abstraídos e delegados a um provedor, de modo que a equipe de desenvolvimento concentra-se na lógica da aplicação. Uma de suas manifestações é o *Backend as a Service* (BaaS), no qual funcionalidades de *backend* comumente necessárias — autenticação, banco de dados, armazenamento de arquivos e autorização — são oferecidas como serviços gerenciados, consumidos diretamente pelo cliente por meio de SDKs e APIs.

Nesse arranjo, parte significativa das regras de negócio e, sobretudo, das regras de autorização desloca-se para a camada de dados. No caso do Supabase, isso se concretiza por meio do *Row Level Security* (RLS) do PostgreSQL, mecanismo que permite definir, de forma **declarativa**, políticas que restringem quais linhas cada usuário pode ler ou modificar, avaliadas pelo próprio banco a cada operação. Complementam o modelo as *views* (que expõem projeções controladas dos dados) e os *triggers* (que aplicam regras de integridade e máquinas de estado no servidor).

Do ponto de vista arquitetural, esse paradigma apresenta implicações relevantes para a avaliação de qualidade:

- A **superfície de autorização** concentra-se em políticas declarativas, cuja correção precisa ser testada explicitamente, pois falhas de RLS podem expor dados sem que haja erro funcional aparente no cliente.
- A **ausência de servidor de aplicação próprio** transfere parte do desempenho percebido para o cliente (carregamento da SPA) e para a latência das chamadas ao serviço gerenciado.
- A **testabilidade** passa a depender da capacidade de testar tanto o cliente quanto as regras residentes no banco.

Essas implicações motivam a necessidade, identificada no problema de pesquisa, de um procedimento de avaliação técnica específico para arquiteturas BaaS/Serverless.

2.5 Engenharia de software: testes e análise estática

Pressman e Maxim (2016) sistematizam o teste de software como atividade planejada e mensurável, distinguindo níveis (unidade, integração, sistema) e abordagens (caixa-branca e caixa-preta), e enfatizam o papel das métricas de software na avaliação objetiva da qualidade de produto e de processo. Sommerville (2011), por sua vez, situa a verificação e a validação, a análise estática e a inspeção de código como práticas complementares ao teste dinâmico, destacando que a análise estática permite detectar classes de defeitos — e indicadores de manutenibilidade, como complexidade e duplicação — sem a execução do programa.

Esses referenciais embasam a escolha das ferramentas e métricas do plano experimental (Seção 5): testes automatizados (de unidade e de integração) para adequação funcional, confiabilidade e testabilidade; análise estática para manutenibilidade; e técnicas específicas para os atributos de segurança e desempenho.

3 METODOLOGIA

3.1 Classificação da pesquisa

A pesquisa é classificada segundo quatro dimensões usuais na metodologia científica: natureza, abordagem, objetivos e procedimentos técnicos.

Tabela 1 — Classificação da pesquisa

Dimensão	Classificação	Justificativa
Natureza	Aplicada	Visa gerar conhecimento de aplicação prática — um procedimento de avaliação técnica — dirigido à solução de um problema concreto de validação arquitetural.
Abordagem	Mista (quantitativa e qualitativa)	Combina a coleta de métricas objetivas (cobertura, tempos de resposta, complexidade, vulnerabilidades) com a análise qualitativa de decisões arquiteturais via ATAM.
Objetivos	Exploratória e descritiva	Explora a aplicação de métodos de avaliação a um paradigma arquitetural pouco coberto pela literatura clássica (BaaS/Serverless) e descreve detalhadamente a arquitetura e os resultados da avaliação.
Procedimentos	Estudo de caso com experimento técnico	Investiga em profundidade um caso real (Hubservi) e conduz um experimento técnico controlado de coleta de métricas e execução de testes.

Fonte: elaborado pelos autores (2026).

3.2 Objeto de estudo

O objeto de estudo é a plataforma **Hubservi**, aplicação web de intermediação de serviços construída como SPA em React e TypeScript e apoiada pelo BaaS Supabase sobre PostgreSQL. A escolha justifica-se por ser um caso representativo do paradigma SPA + BaaS + Serverless, com regras de negócio expressivas residentes no banco (políticas de RLS, *triggers* e *views*), o que a torna adequada à investigação do problema de pesquisa. A arquitetura do objeto é detalhada na Seção 4.

3.2.1 Construção do instrumento

Conforme a delimitação da Seção 1.5, a plataforma é o **instrumento** da avaliação, desenvolvido integralmente pela equipe. Sua construção seguiu um processo estruturado, guiado por artefatos de Engenharia de Software, e não é objeto de avaliação em si — registra-se aqui por dois motivos: a **validade interna** do experimento depende de o instrumento ser conhecido e controlado, e parte desses artefatos alimenta diretamente as etapas seguintes da pesquisa.

- **Iniciação e concepção:** Termo de Abertura do Projeto (TAP), Modelo Canvas e análise SWOT delimitaram problema, escopo (MVP), premissas, restrições e critérios de sucesso.

- **Planejamento:** Estrutura Analítica do Projeto (EAP) e 5W2H organizaram as entregas; os requisitos foram especificados e priorizados sob identificadores rastreáveis (RF-xx funcionais, RNF-xx não funcionais).
- **Modelagem:** UML (casos de uso, classes, componentes, sequência e implantação), BPMN (contratação e gerenciamento de *booking*) e modelo de dados (DER e dicionário de dados).
- **Construção incremental:** entrega por marcos — **M1** requisitos e arquitetura; **M2** fluxos centrais (autenticação, serviços, *booking*); **M3** segurança e governança de dados (RLS e *triggers*); **M4** validação de qualidade; **M5** evolução da descoberta de serviços (ordenação e filtros).
- **Fluxo de qualidade contínuo:** *lint*, testes e revisão a cada alteração, conforme premissa registrada no TAP.

Os artefatos de modelagem, além de documentarem o instrumento, **atendem aos objetivos específicos 2 e 3** e constituem a base a partir da qual se derivam a árvore de utilidade do ATAM (Seção 6) e os cenários de avaliação (Seção 5) — ou seja, ligam a construção do instrumento à avaliação propriamente dita. Encontram-se reproduzidos no material suplementar (VIEIRA; COSTA, 2026a).

3.3 Etapas metodológicas

A condução do trabalho organiza-se em seis etapas, alinhadas aos objetivos específicos (Seção 1.4.2):

1. **Levantamento de requisitos arquiteturais e atributos de qualidade.** Identificação dos *Architecturally Significant Requirements* (ASR) e seleção das características de qualidade da ISO/IEC 25010 (2011) relevantes ao domínio: segurança, eficiência de desempenho, manutenibilidade (com ênfase em testabilidade) e confiabilidade.
2. **Modelagem da arquitetura.** Recuperação e representação da arquitetura real a partir do código-fonte e das *migrations*, produzindo modelos UML (casos de uso, classes, sequência, componentes e implantação), modelo de processos de negócio (BPMN) e o modelo de dados (DER e dicionário de dados). Os artefatos constam do material suplementar (VIEIRA; COSTA, 2026a).
3. **Documentação técnica.** Descrição dos componentes, fluxos, persistência e regras de negócio (Seção 4), assegurando rastreabilidade entre cada afirmação e sua fonte no repositório.
4. **Definição de cenários de avaliação.** Construção de uma árvore de utilidade ATAM e especificação de cenários de atributo de qualidade no formato de seis partes (Seção 6), além do mapeamento atributo, cenário, métrica, critério e ferramenta (Seção 5).
5. **Execução da avaliação técnica.** Execução de testes automatizados (unitários, de componente e de integração contra a API), análise estática de código, testes de segurança e testes de desempenho, conforme o plano de métricas. O ambiente de teste de banco/API foi levantado com uma instância Supabase local, garantindo reprodutibilidade e isolamento em relação a dados de produção.
6. **Análise dos resultados.** Interpretação das métricas coletadas à luz dos critérios definidos e dos *tradeoffs* identificados no ATAM, com discussão dos achados e recomendações (Seção 7).

3.4 Instrumentos e ferramentas

A coleta de dados apoia-se em um conjunto de ferramentas, organizado por atributo de qualidade e detalhado na Seção 5. Onde a ferramenta inicialmente prevista não estava disponível no ambiente (por exigir conta ou instalação indisponível), adotou-se um substituto **da mesma classe**, medindo as mesmas métricas; cada substituição está registrada no registro de medições (<https://github.com/Richardy-Rodrigues/hubservi/blob/tcc-v2/docs/tcc/medicoes/registro-medicoes.md>).

- **Testes automatizados:** Vitest e React Testing Library (unitários e de componente); Vitest contra o *stack* Supabase local via PostgREST (integração).
- **Análise estática:** ESLint; *eslint-plugin-sonarjs* para *code smells* e complexidade — em substituição ao SonarQube, que exige serviço externo; *jscpd* para duplicação.
- **Análise de dependências e modularização:** Madge e dependency-cruiser.
- **Desempenho:** Lighthouse (carregamento inicial); autocannon (carga sobre a API) — em substituição ao k6/JMeter.
- **Segurança:** *npm audit* para análise de composição de dependências (SCA) — em substituição ao Snyk, que exige conta; *supabase db lint* para o *schema*. A varredura dinâmica (DAST/OWASP ZAP) permanece pendente de execução contra a URL de produção.

3.5 Procedimentos de análise

As métricas quantitativas são confrontadas com os critérios de aceitação previamente definidos (limiares e metas), e os achados qualitativos organizados segundo os conceitos do ATAM (riscos, pontos de sensibilidade e pontos de compromisso). A triangulação entre as evidências quantitativas e a análise arquitetural qualitativa sustenta a resposta à questão de pesquisa.

Adotou-se, na execução, o ciclo **medir, detectar, corrigir e re-medir**: quando uma medição reprova o sistema, o defeito é corrigido e a medição repetida, reportando-se o par antes/depois. Um conjunto de cenários que aprovasse o sistema já na primeira execução não demonstraria qualidade, e sim fraqueza dos cenários — por isso os dois estados são preservados como evidência.

Regra de registro e anti-fabricação. Toda medição registra ferramenta e versão, ambiente, configuração, data, valor obtido e veredito, com o artefato bruto correspondente arquivado publicamente (<https://github.com/Richardy-Rodrigues/hubservi/tree/tcc-v2/docs/tcc/medicoes/evidencias>) e reproduzido como captura de tela no Apêndice C (VIEIRA; COSTA, 2026c). Nenhum valor é reportado sem evidência associada, e os resultados desfavoráveis são registrados com o mesmo rigor que os favoráveis (Seção 7).

4 ARQUITETURA DO HUBSERVI

Esta seção documenta a arquitetura real da plataforma Hubservi, recuperada a partir do código-fonte (*src/*) e das *migrations* do banco de dados (*supabase/migrations/*). A descrição responde aos objetivos específicos 2 e 3 (modelar e documentar componentes, fluxos, persistência e regras de negócio).

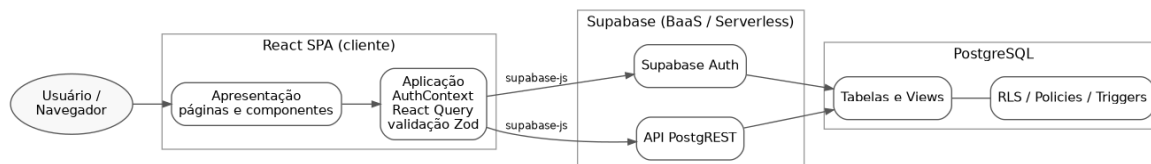
4.1 Visão geral e estilo arquitetural

O Hubservi adota o estilo **SPA + BaaS + Serverless**. Um *Single Page Application* (SPA) executado no navegador concentra a interface e a orquestração da lógica de aplicação e comunica-se diretamente com o BaaS Supabase, que provê autenticação, banco de dados PostgreSQL e autorização declarativa via *Row Level Security* (RLS). Não há, portanto, servidor de aplicação intermediário desenvolvido sob medida, tampouco decomposição em

microsserviços: as responsabilidades de *backend* são delegadas a serviços gerenciados, e parte expressiva das regras de negócio reside no próprio banco, sob a forma de *triggers*, *views* e políticas de RLS.

A arquitetura organiza-se em camadas lógicas, conforme a Figura 1.

Figura 1 — Camadas lógicas da arquitetura do Hubservi



Fonte: elaborado pelos autores (2026).

Tabela 2 — Camadas lógicas e elementos correspondentes no repositório

Camada	Responsabilidade	Elementos no repositório
Apresentação	Renderização, navegação e interação	<i>src/pages/, src/components/, src/components/ui/</i>
Aplicação	Sessão, estado de servidor, regras de fluxo e validação	<i>src/contexts/AuthContext.tsx</i> , React Query, esquemas Zod
Integração	Acesso ao BaaS	<i>src/integrations/supabase/client.ts</i> , <i>views.ts</i> , <i>types.ts</i>
Dados	Persistência e regras declarativas	<i>supabase/migrations/</i> (PostgreSQL)

Fonte: elaborado pelos autores (2026).

Os diagramas detalhados de componentes e de implantação constam do material suplementar (VIEIRA; COSTA, 2026a, Figuras A.7 e A.8).

4.2 Tecnologias

Tabela 3 — Tecnologias empregadas na plataforma Hubservi

Camada	Tecnologia	Versão
Biblioteca de UI	React	18.3.1
Linguagem	TypeScript	5.8.3
Empacotador / <i>build</i>	Vite	5.4.19
Roteamento	React Router DOM	6.30.1
Estado de servidor	TanStack React Query	5.83.0
Formulários e validação	react-hook-form + Zod	7.61 / 3.25
Componentes de UI	shadcn/ui (Radix UI) + Tailwind CSS	3.4.17
Cliente BaaS	@supabase/supabase-js	2.98.0
Banco de dados	PostgreSQL (gerenciado pelo Supabase)	—

Fonte: elaborado pelos autores (2026).

4.3 Componentes e rotas

A aplicação define as rotas apresentadas na Tabela 4 (*src/App.tsx*).

Tabela 4 — Rotas da aplicação e respectivo nível de acesso

Rota	Página	Acesso
/	<i>Index.tsx</i> — landing page	Público
/auth	<i>Auth.tsx</i> — login e cadastro	Público
/services	<i>Services.tsx</i> — busca e listagem	Público
/services/:id	<i>ServiceDetail.tsx</i> — detalhe, avaliações e solicitação	Público
/dashboard	<i>Dashboard.tsx</i> — painel por perfil	Protegido
*	<i>NotFound.tsx</i> — 404	Público

Fonte: elaborado pelos autores (2026).

O acesso à rota protegida é controlado pelo componente *ProtectedRoute.tsx*, que redireciona usuários sem sessão para */auth*. O *Dashboard.tsx* seleciona a interface conforme o tipo de usuário: *ClientDashboard* (cliente) ou *ProviderDashboard* (prestador). Provedores globais configurados na raiz incluem *QueryClientProvider* (cache e sincronização de dados), *AuthProvider* (sessão e perfil), *BrowserRouter* e provedores de *feedback* de interface. Os diagramas de casos de uso e de classes constam do material suplementar (VIEIRA; COSTA, 2026a, Figuras A.2 e A.3).

4.4 Modelo de dados

O esquema do banco compreende cinco tabelas, três tipos enumerados e duas *views*.

Tabelas: *profiles*, *categories*, *services*, *bookings*, *reviews*.

Enumerações:

- *user_type*: *client*, *provider*;
- *price_type*: *fixed*, *hourly*, *negotiable*;
- *booking_status*: *pending*, *accepted*, *completed*, *rejected*, *cancelled*.

Views:

- *service_stats* — agrega, por serviço, a contagem de avaliações (*review_count*) e a média de notas (*average_rating*); definida com *security_invoker* = *true*.
- *public_profiles* — projeção de *profiles* sem dados pessoais sensíveis (expõe *id*, *full_name*, *avatar_url*, *user_type*, *created_at*; **omite** *email* e *phone*), utilizada para apresentar dados de perfil a usuários não autenticados.

As chaves estrangeiras estabelecem as relações: *services* referencia *profiles* (prestador) e *categories*; *bookings* referencia *services* e duas vezes *profiles* (cliente e prestador); *reviews* referencia *services* e *profiles* (cliente e prestador). A tabela *reviews* possui a restrição de unicidade (*service_id*, *client_id*), garantindo no máximo uma avaliação por cliente por serviço. O DER e o dicionário de dados constam do material suplementar (VIEIRA; COSTA, 2026a, Figura A.1 e Tabelas A.1 a A.7).

4.5 Fluxos principais

4.5.1 Autenticação

O cadastro (*Auth.tsx*) chama *supabase.auth.signUp*, fornecendo *full_name* e *user_type* em metadados. O *trigger on_auth_user_created* materializa, de forma idempotente, a linha correspondente em *profiles*. O *AuthContext* assina *onAuthStateChange* e carrega o perfil do usuário autenticado (VIEIRA; COSTA, 2026a, Figura A.4).

4.5.2 Cadastro e busca de serviços

Prestadores criam, editam e removem serviços pelo *ServiceForm* no *ProviderDashboard*, com validação *Zod*. A busca (*Services.tsx*) consulta serviços ativos (*is_active = true*), com filtro por categoria e por título, paginação e ordenação por **recência**, **avaliação** ou **popularidade** — estas duas últimas apoiadas na *view_service_stats*. Cabe destacar que a recomendação de serviços, no *Hubservi*, resume-se a essa ordenação por popularidade e avaliação, constituindo módulo secundário e não o foco deste trabalho.

4.5.3 Contratação (booking)

A solicitação (*BookingDialog*) insere um registro em *bookings* com *status = 'pending'*. O cliente acompanha e pode cancelar solicitações pendentes; o prestador aceita, rejeita, conclui ou cancela (VIEIRA; COSTA, 2026a, Figuras A.5 e A.9).

4.5.4 Avaliação (review)

Concluído um *booking*, o cliente pode registrar uma avaliação (nota de 1 a 5 e comentário) pelo *ReviewForm* (VIEIRA; COSTA, 2026a, Figura A.6).

4.6 Regras de negócio residentes no banco

Boa parte das invariantes do domínio é imposta no servidor por *triggers* e restrições, e não apenas no cliente — característica marcante do paradigma BaaS.

Tabela 5 — Regras de negócio implementadas no banco de dados

Regra	Mecanismo	Fonte
Provisão automática de perfil no cadastro	<i>trigger on_auth_user_created</i> para <i>handle_new_user()</i> (idempotente, <i>SECURITY DEFINER</i>)	<i>migration</i> inicial; 20260316201000
<i>updated_at</i> atualizado a cada alteração	<i>trigger update_updated_at_column()</i>	<i>migration</i> inicial
Máquina de estados do <i>booking</i>	<i>trigger validate_booking_status_transition()</i>	<i>migration</i> inicial; 20260528000000
Imutabilidade do <i>user_type</i> (anti-escalonamento de privilégio)	<i>trigger prevent_user_type_change()</i>	20260514100000
<i>booking.provider_id</i> deve coincidir com o dono do serviço	<i>trigger validate_booking_provider()</i>	20260514100100
<i>price_max</i> maior ou igual a <i>price_min</i>	<i>constraint services_price_range_check</i>	20260514100300
Avaliação só após <i>booking completed</i>	política RLS de <i>INSERT</i> em <i>reviews</i>	20260514100400
Cliente pode cancelar <i>booking</i> pendente	política RLS e transição <i>pending</i> para <i>cancelled</i>	20260528000000
<i>review.provider_id</i> deve coincidir com o dono do serviço	<i>trigger</i> de validação espelhado (correção reportada em 7.3.2)	<i>migration</i> de correção (julho de 2026)

Fonte: elaborado pelos autores (2026).

A máquina de estados do *booking* admite as transições *pending* para *accepted*, *rejected* ou *cancelled*, e *accepted* para *completed* ou *cancelled*; transições inválidas resultam em exceção (VIEIRA; COSTA, 2026a, Figura A.10).

4.7 Segurança: autorização declarativa via RLS

Todas as tabelas têm RLS habilitado. A autorização é expressa por políticas declarativas avaliadas pelo PostgreSQL a cada operação. A Tabela 6 resume as políticas vigentes, já **no estado posterior às correções** de segurança reportadas na Seção 7.3.2.

Tabela 6 — Políticas de Row Level Security por tabela e operação

Tabela	Operação	Política (resumo)
<i>profiles</i>	SELECT/UPDATE/INSERT	usuário acessa e edita apenas o próprio perfil (<i>auth.uid() = id</i>); dados de contraparte são obtidos pela <i>view public_profiles</i> , sem PII
<i>categories</i>	SELECT	leitura pública
<i>services</i>	SELECT	qualquer um vê serviços ativos; prestador vê os próprios (ativos ou não)
<i>services</i>	INSERT/UPDATE/DELETE	apenas o prestador dono (<i>auth.uid() = provider_id</i>)
<i>bookings</i>	SELECT	cliente vê os próprios; prestador vê os próprios
<i>bookings</i>	INSERT	apenas o cliente (<i>auth.uid() = client_id</i>)
<i>bookings</i>	UPDATE	prestador altera status; cliente pode cancelar apenas os próprios pendentes
<i>reviews</i>	SELECT	leitura pública
<i>reviews</i>	INSERT	cliente com <i>booking completed</i> no serviço; <i>provider_id</i> validado por <i>trigger</i>
<i>reviews</i>	UPDATE/DELETE	apenas o autor (<i>auth.uid() = client_id</i>)

Fonte: elaborado pelos autores (2026).

A proteção de dados pessoais (e-mail e telefone) é assegurada pela restrição da leitura direta de *profiles* ao próprio registro, combinada à *view public_profiles* para consumo por terceiros e por usuários anônimos. Registre-se que essa configuração é resultado da avaliação: a política originalmente vigente admitia leitura por qualquer usuário autenticado (*USING (auth.uid() IS NOT NULL)*), defeito detectado e corrigido conforme a Seção 7.3.2. Esse arranjo — autorização concentrada em políticas declarativas — é precisamente o ponto que a avaliação de segurança (Seções 5 e 6) exercitou de forma sistemática.

4.8 Síntese arquitetural

O Hubservi exemplifica os *tradeoffs* característicos do paradigma BaaS/Serverless: ganha-se simplicidade operacional e velocidade de desenvolvimento ao delegar autenticação, persistência e autorização a serviços gerenciados, ao custo de concentrar a correção da segurança em configurações declarativas e de transferir parte do desempenho percebido para o cliente e para a latência das chamadas ao serviço. Esses pontos orientaram o planejamento experimental apresentado a seguir.

5 PLANEJAMENTO EXPERIMENTAL

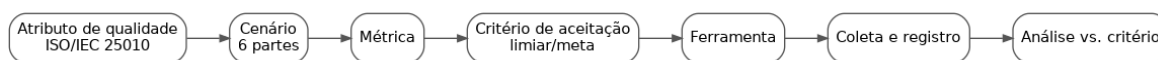
Esta seção operacionaliza a avaliação técnica, atendendo aos objetivos específicos 4 a 8. Para cada atributo de qualidade da ISO/IEC 25010 (2011) considerado relevante, definiu-se

um conjunto de cenários, métricas, critérios de aceitação e ferramentas. Os critérios aqui apresentados são **metas e limiares de planejamento**, fixados antes da coleta; os valores efetivamente medidos constam da Seção 7.

5.1 Visão geral do procedimento de medição

O procedimento encadeia atributo de qualidade, cenário em seis partes, métrica, critério de aceitação, ferramenta, coleta e registro, e análise em face do critério, conforme a Figura 2.

Figura 2 — Procedimento de medição adotado



Fonte: elaborado pelos autores (2026).

Cada execução de medição registrou: ferramenta e versão, ambiente, configuração, data, valor obtido e veredito (atende ou não atende ao critério). A reprodutibilidade foi assegurada pela fixação de ambiente e parâmetros, conforme a etapa 5 da metodologia (Seção 3.3).

5.2 Mapeamento entre atributo, cenário, métrica, critério e ferramenta

Os valores da coluna **Critério (meta)** são alvos de planejamento; não representam resultados medidos. A coluna **Ferramenta** já registra os substitutos efetivamente utilizados, conforme a Seção 3.4.

5.2.1 Segurança

Tabela 7 — Cenários, métricas e critérios de segurança

Cenário	Métrica	Critério (meta)	Ferramenta
Usuário tenta ler ou alterar dados de outro usuário (<i>booking</i> , perfil, serviço)	Nº de acessos indevidos bem-sucedidos	0 acessos indevidos	Testes de política RLS contra a API
Cliente tenta avaliar serviço sem <i>booking</i> concluído	Nº de inserções de <i>review</i> indevidas	0 inserções	Teste de integração contra a API
Usuário tenta alterar o próprio <i>user_type</i> (escalonamento)	Tentativa bloqueada (sim/não)	100% bloqueadas	Teste de integração e verificação do <i>trigger</i>
Varredura de vulnerabilidades da aplicação web	Nº de vulnerabilidades por severidade	0 de severidade alta ou crítica	OWASP ZAP (pendente — ver 7.3.2)
Vulnerabilidades em dependências	Nº de CVEs por severidade	0 de severidade alta ou crítica não tratadas	<i>npm audit</i> (em substituição ao Snyk)
Exposição de PII a usuário anônimo e a usuário autenticado não relacionado	Campos sensíveis expostos (<i>email</i> , <i>phone</i>)	0 campos expostos a ambos os perfis de acesso	Teste de política RLS e inspeção da <i>view public_profiles</i>
Conformidade do <i>schema</i> do banco	Nº de erros de <i>lint</i> de <i>schema</i>	0 erros	<i>supabase db lint</i>

Fonte: elaborado pelos autores (2026).

Nota de refinamento do cenário de exposição de PII. Em sua formulação inicial, este cenário considerava apenas o acesso **anônimo**. A inspeção das políticas vigentes evidenciou que a política de leitura de *profiles* admitia qualquer usuário autenticado (*USING (auth.uid()) IS NOT NULL*)), de modo que o cenário restrito ao anônimo seria satisfeito sem exercitar o perfil

de acesso mais permissivo efetivamente existente. O cenário foi, por isso, ampliado para contemplar também o **usuário autenticado não relacionado ao perfil consultado**. Registra-se o ajuste por transparência metodológica: a formulação de cenários é atividade sujeita a refinamento à luz da arquitetura concreta sob avaliação, e um cenário incapaz de reprovar o sistema não constitui instrumento de medição.

5.2.2 Eficiência de desempenho

Tabela 8 — Cenários, métricas e critérios de eficiência de desempenho

Cenário	Métrica	Critério (meta)	Ferramenta
Carregamento inicial da SPA	<i>Performance score</i> ; LCP; TBT; CLS	Score maior ou igual a 90; LCP menor ou igual a 2,5 s	Lighthouse
Listagem e busca de serviços sob carga	Tempo de resposta (p95); vazão (req/s); taxa de erro	p95 menor ou igual a 800 ms; erro abaixo de 1%	autocannon (em substituição ao k6)
Operações de <i>booking</i> sob carga concorrente	Latência (p95); taxa de erro	p95 menor ou igual a 800 ms; erro abaixo de 1%	autocannon (em substituição ao k6/JMeter)
Comportamento sob carga sustentada	Estabilidade de latência e de erros ao longo do tempo	Sem degradação progressiva	autocannon (em substituição ao JMeter)

Fonte: elaborado pelos autores (2026).

O limiar de 800 ms para a latência p95, em aberto no planejamento inicial, foi calibrado antes da coleta definitiva a partir de uma execução exploratória de baixa intensidade (5 conexões por 5 s, latência de cauda aproximada de 57 ms) e da heurística de 1 s para a percepção de fluidez pelo usuário, adotando-se margem conservadora.

Percentil efetivamente reportado. A ferramenta empregada (autocannon) não expõe o p95 no conjunto padrão de percentis de seu histograma — os valores vizinhos são o p90 e o p97,5. Os resultados de latência sob carga apresentados na Seção 7 são, portanto, de **p97,5**, percentil mais exigente que o p95 fixado como critério: satisfeito o limiar em p97,5, ele está necessariamente satisfeito em p95. Registra-se a diferença para que o número publicado não prometa precisão superior à que o instrumento entrega.

5.2.3 Testabilidade (subcaracterística de manutenibilidade)

Tabela 9 — Cenários, métricas e critérios de testabilidade

Cenário	Métrica	Critério (meta)	Ferramenta
Cobertura da suíte de testes	Cobertura de linhas e de ramos (%)	Piso global anti-regressão de 30% e cobertura maior ou igual a 75% nos módulos críticos (limiar definido na execução — ver 7.3.4)	Vitest (<i>coverage</i> v8)
Isolamento de componentes em teste	Nº de componentes testáveis sem dependências externas reais (uso de <i>mocks</i>)	Fluxos críticos cobertos por teste isolado	Vitest e React Testing Library
Esforço de criação de teste por fluxo crítico	Existência de teste para cada fluxo crítico	100% dos fluxos críticos com ao menos um teste	Vitest e React Testing Library

Cenário	Métrica	Critério (meta)	Ferramenta
	(autenticação, <i>booking</i> , <i>review</i>)		

Fonte: elaborado pelos autores (2026).

5.2.4 Manutenibilidade

Tabela 10 — Cenários, métricas e critérios de manutenibilidade

Cenário	Métrica	Critério (meta)	Ferramenta
Conformidade de estilo e antipadrões	Nº de violações de <i>lint</i>	Tendência a 0; sem erros	ESLint
Complexidade e <i>code smells</i>	Complexidade ciclomática; densidade de <i>code smells</i>	Dentro dos limiares das regras adotadas	<i>eslint-plugin-sonarjs</i> (em substituição ao SonarQube)
Duplicação de código	% de linhas duplicadas	Menor ou igual a 3%	<i>jscpd</i> (em substituição ao SonarQube)
Modularização e acoplamento	Nº de dependências circulares; grafo de dependências	0 ciclos	Madge e dependency-cruiser

Fonte: elaborado pelos autores (2026).

5.2.5 Confiabilidade

Tabela 11 — Cenários, métricas e critérios de confiabilidade

Cenário	Métrica	Critério (meta)	Ferramenta
Consistência da máquina de estados do <i>booking</i>	Nº de transições inválidas aceitas	0 transições inválidas	Teste de integração do <i>trigger</i>
Integridade referencial em exclusões em cascata	Nº de registros órfãos após exclusão	0 órfãos	Teste de integração
Fluxos críticos ponta a ponta (autenticação, <i>booking</i> , <i>review</i>)	Taxa de sucesso dos fluxos	100% nos casos válidos	Testes de integração

Fonte: elaborado pelos autores (2026).

5.3 Ferramentas: papel e disponibilidade

A Tabela 12 consolida as ferramentas efetivamente empregadas na execução, registrando as substituições em relação ao plano inicial. Todas as trocas preservaram a classe de ferramenta e as métricas coletadas, conforme a Seção 3.4.

Tabela 12 — Ferramentas empregadas e substituições em relação ao plano inicial

Categoria	Ferramenta empregada	Prevista no plano inicial	Situação
Teste automatizado	Vitest, React Testing Library	Vitest, React Testing Library	Utilizada; suíte ampliada na execução
Teste de integração	Vitest contra Supabase local via PostgREST	Testes contra a API	Utilizada
Análise estática	ESLint (<i>flat config</i> , ESLint 9)	ESLint	Utilizada; complementada com regras de qualidade

Categoria	Ferramenta empregada	Prevista no plano inicial	Situação
Complexidade e <i>code smells</i>	<i>eslint-plugin-sonarjs</i>	SonarQube	Substituída (SonarQube exige serviço externo)
Duplicação	<i>jscpd</i>	SonarQube	Substituída
Dependências e modularização	Madge, dependency-cruiser	Madge, dependency-cruiser	Utilizadas
Desempenho (carregamento)	Lighthouse	Lighthouse	Utilizada sobre o <i>build</i> de produção
Desempenho (carga)	autocannon	k6, JMeter	Substituída
Segurança (SCA)	<i>npm audit</i>	Snyk	Substituída (Snyk exige conta)
Segurança (<i>schema</i>)	<i>supabase db lint</i>	—	Acrescentada na execução
Segurança (DAST)	OWASP ZAP	OWASP ZAP	Pendente — depende da URL de produção

Fonte: elaborado pelos autores (2026).

5.4 Ambiente experimental

Uma medição só é reproduzível se o ambiente em que ocorreu for conhecido. A Tabela 13 caracteriza o ambiente único em que toda a coleta foi realizada, nas datas de 15 e 16 de julho de 2026, sobre o estado do repositório fixado pelo *commit ad89e6c*. Os testes de desempenho de carregamento utilizaram o *build* de produção (*vite build*); as medições de segurança, de autorização (RLS) e de confiabilidade ocorreram contra uma instância Supabase local em contêiner, reconstruída do zero a cada execução por *supabase db reset*, o que elimina qualquer contato com dados reais de produção. Os valores da tabela são transcritos dos arquivos *ambiente.txt* arquivados com as evidências (<https://github.com/Richardy-Rodrigues/hubservi/tree/tcc-v2/docs/tcc/medicoes/evidencias>).

Tabela 13 — Caracterização do ambiente experimental

Dimensão	Configuração registrada
Host	Microsoft Windows 11 Pro for Workstations, <i>build</i> 26200; processador AMD Ryzen 5 PRO 230; 15,2 GB de memória RAM
Tempo de execução	Node.js 24.14.0; npm 11.9.0 (versão fixada em <i>packageManager</i>); dependências instaladas por <i>npm ci</i> , a partir do <i>package-lock.json</i>
Aplicação avaliada	<i>Build</i> de produção gerado por Vite 5.4.19; servido localmente para as execuções do Lighthouse
Banco de dados e API	Supabase CLI 2.109.1, invocada por <i>npx</i> com versão fixada; imagem de contêiner <i>supabase/postgres:15.8.1.085</i> (PostgreSQL 15.8), executada em Docker Desktop sobre WSL 2; API PostgREST exposta em <i>127.0.0.1:54321</i>
Versões das ferramentas (Tabela 12)	Vitest 3.2.7 com <i>@vitest/coverage-v8</i> ; ESLint 9.32 com <i>typescript-eslint</i> 8.38 e <i>eslint-plugin-sonarjs</i> 3; <i>jscpd</i> 4; Madge 8; dependency-cruiser 16.10.4; Lighthouse 12.8.2 (perfil móvel, com limitação de CPU e de rede); autocannon 8; <i>npm audit</i> ; <i>supabase db lint</i> 2.109.1

Dimensão	Configuração registrada
Volume de dados	Base reconstruída do zero (10 <i>migrations</i> e <i>seed</i>), com 11 categorias e nenhum usuário pré-existente; as suítes de integração criam e descartam os próprios usuários e registros a cada execução; o teste de carga foi semeado com 50 serviços ativos de um prestador, consultados com <i>limit=20</i> por requisição

Fonte: elaborado pelos autores (2026), a partir dos registros de ambiente das coletas de 15 e 16 de julho de 2026.

Uma dimensão do ambiente **não foi registrada** na coleta: a versão do Docker Desktop. Registra-se a omissão em vez de preenchê-la com a versão instalada hoje, que não seria a da medição. O impacto é considerado baixo, porque o que determina o comportamento do banco é a imagem de contêiner, essa sim fixada por *tag* (15.8.1.085), e não o programa que a executa.

5.5 Ameaças à validade

- **Ambiente local em vez de produção (validade externa).** As medições de API e de banco correm contra a instância local, sem latência de rede real nem os limites do plano de serviço gerenciado. Os números de desempenho de *backend* devem ser lidos como um **piso** — uma execução contra a instância gerenciada tende a apresentar latência maior.
- **Volume de dados reduzido (validade de construção).** O teste de carga exercita uma tabela com 50 serviços, muito abaixo de qualquer operação real. O resultado atesta que a camada de dados responde sob concorrência, não que se mantenha sob volume; latência de leitura é sensível ao tamanho da relação e à seletividade dos índices, e essa dimensão permanece fora do escopo desta avaliação.
- **Máquina única e não dedicada (validade de conclusão).** Toda a coleta ocorreu em um só *host*, compartilhado com o sistema operacional do usuário. Mitigou-se pela repetição das execuções sensíveis a ruído — o Lighthouse é reportado pela mediana de três execuções — e pela fixação dos parâmetros de cada ferramenta.
- **Representatividade dos cenários (validade de construção).** Os cenários derivam da árvore de utilidade do ATAM (Seção 6) e não de dados de uso real, inexistentes para uma plataforma ainda não operada em produção.
- **Avaliação conduzida pela equipe desenvolvedora (viés do avaliador).** Mitigou-se pela fixação dos critérios de aceitação **antes** da coleta (Seção 5.2) e pela regra de registro anti-fabricação (Seção 3.5), que obriga a reportar o resultado desfavorável com o mesmo rigor do favorável.

Nem toda medição planejada é igualmente reproduzível por terceiros, e o Apêndice B (VIEIRA; COSTA, 2026b) classifica cada uma quanto a isso, explicitando as que **não** reproduzem e por quê — a cobertura do *baseline*, por depender de um estado de árvore anterior às correções, e as medições de tempo, que reproduzem a faixa e o veredito, não o valor exato.

6 AVALIAÇÃO ARQUITETURAL COM ATAM

Esta seção apresenta a aplicação do *Architecture Tradeoff Analysis Method* (ATAM) à arquitetura do Hubservi, conforme Clements, Kazman e Klein (2002). O ATAM complementa o plano de métricas (Seção 5): enquanto este coleta evidências quantitativas, o ATAM organiza a análise qualitativa das decisões arquiteturais, evidenciando riscos, pontos de sensibilidade e pontos de compromisso.

6.1 Etapas do ATAM aplicadas ao estudo de caso

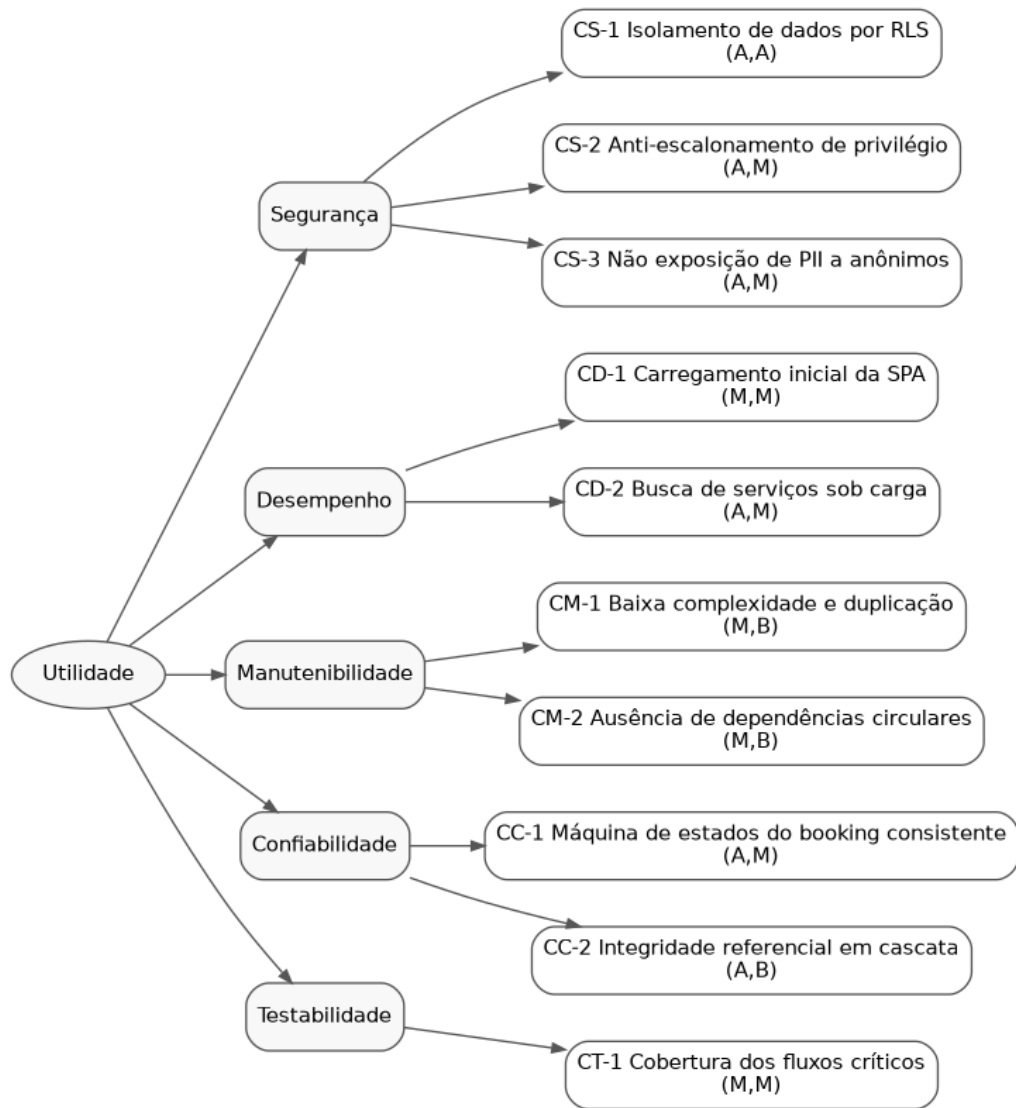
O método foi conduzido nas etapas a seguir, adaptadas ao contexto de um estudo de caso conduzido pela própria equipe técnica:

1. **Apresentação do ATAM** — alinhamento do método com os envolvidos.
2. **Apresentação dos direcionadores de negócio** — intermediação de serviços, confiança entre partes e proteção de dados pessoais.
3. **Apresentação da arquitetura** — conforme a Seção 4 (SPA + BaaS + Serverless).
4. **Identificação das abordagens arquiteturais** — delegação de *backend* ao BaaS; autorização declarativa por RLS; regras de negócio em *triggers* e *views*; estado de servidor gerenciado por React Query no cliente.
5. **Construção da árvore de utilidade** — Seção 6.2.
6. **Análise das abordagens arquiteturais** — Seção 6.4.
7. **Brainstorming e priorização de cenários** — refinamento dos cenários da árvore de utilidade.
8. **Reanálise das abordagens** — à luz dos cenários priorizados e das métricas coletadas.
9. **Apresentação dos resultados** — riscos, temas de risco e *tradeoffs* consolidados, reportados na Seção 6.4 com apoio das medições da Seção 7.

6.2 Árvore de utilidade

A árvore decompõe a utilidade geral em atributos de qualidade, refinamentos e cenários, cada qual rotulado por um par (**importância para o negócio, risco arquitetural**) em escala alto, médio e baixo — A, M e B (Figura 3).

Figura 3 — Árvore de utilidade dos cenários de atributo de qualidade



Fonte: elaborado pelos autores (2026).

Os cenários priorizados são: **CS-1** isolamento de dados por RLS (A, A); **CS-2** anti-escalonamento de privilégio (A, M); **CS-3** não exposição de PII a anônimos (A, M); **CD-1** carregamento inicial da SPA (M, M); **CD-2** busca de serviços sob carga (A, M); **CM-1** baixa complexidade e duplicação (M, B); **CM-2** ausência de dependências circulares (M, B); **CC-1** máquina de estados do *booking* consistente (A, M); **CC-2** integridade referencial em cascata (A, B); e **CT-1** cobertura dos fluxos críticos (M, M).

6.3 Cenários de atributo de qualidade (formato de seis partes)

Os cenários prioritários são especificados segundo a estrutura de Bass, Clements e Kazman (2012): fonte, estímulo, artefato, ambiente, resposta e medida.

CS-1 — Isolamento de dados por RLS. *Fonte:* usuário autenticado mal-intencionado. *Estímulo:* requisição para ler ou alterar registros de outro usuário. *Artefato:* políticas RLS das tabelas *bookings*, *services* e *profiles*. *Ambiente:* operação normal. *Resposta:* a requisição é negada pelo banco. *Medida:* 0 acessos indevidos bem-sucedidos.

CS-2 — Anti-escalonamento de privilégio. *Fonte:* usuário autenticado. *Estímulo:* tentativa de alterar o próprio *user_type*. *Artefato:* *trigger prevent_user_type_change()*. *Ambiente:* operação normal. *Resposta:* alteração rejeitada com exceção. *Medida:* 100% das tentativas bloqueadas.

CS-3 — Não exposição de PII. *Fonte:* visitante não autenticado e usuário autenticado não relacionado. *Estímulo:* leitura de dados de perfil. *Artefato:* `view_public_profiles` e RLS de `profiles`. *Ambiente:* operação normal. *Resposta:* apenas campos não sensíveis são retornados. *Medida:* 0 ocorrências de `email` ou `phone` expostos.

CD-2 — Busca de serviços sob carga. *Fonte:* conjunto de usuários concorrentes. *Estímulo:* requisições simultâneas de busca e listagem. *Artefato:* API `PostgreSQL` e `view_service_stats`. *Ambiente:* carga de pico planejada. *Resposta:* respostas corretas dentro do tempo-alvo. *Medida:* p95 menor ou igual a 800 ms; taxa de erro abaixo de 1%.

CC-1 — Máquina de estados do *booking* consistente. *Fonte:* cliente ou prestador. *Estímulo:* tentativa de transição de status, válida ou inválida. *Artefato:* `trigger_validate_booking_status_transition()`. *Ambiente:* operação normal. *Resposta:* transições válidas aplicadas e inválidas rejeitadas. *Medida:* 0 transições inválidas aceitas.

Os demais cenários (CD-1, CM-1, CM-2, CC-2 e CT-1) seguem a mesma estrutura e estão associados às métricas da Seção 5.

6.4 Pontos de sensibilidade, de compromisso e riscos

A análise a seguir derivou da inspeção arquitetural (Seção 4) e foi confirmada pelas medições da Seção 7.

Tabela 14 — Pontos de sensibilidade, pontos de compromisso, riscos e não riscos identificados

Tipo	Descrição
Ponto de sensibilidade	A correção da segurança é altamente sensível à completude e exatidão das políticas de RLS e dos <i>triggers</i> : uma política ausente ou mal especificada compromete CS-1 e CS-3 sem gerar erro funcional aparente. Os dois defeitos reportados na Seção 7.3.2 confirmam empiricamente essa sensibilidade.
Ponto de sensibilidade	O desempenho de CD-2 é sensível à eficiência da <code>view_service_stats</code> e à indexação das tabelas consultadas.
Ponto de compromisso	A delegação total de autorização ao banco (RLS) favorece simplicidade e consistência (positivo para manutenibilidade e segurança), mas concentra o risco em um único mecanismo declarativo e desloca o esforço de teste para a fronteira cliente-banco (impacto em testabilidade).
Ponto de compromisso	A arquitetura SPA reduz acoplamento de <i>backend</i> e acelera o desenvolvimento, porém transfere carga de processamento e desempenho percebido para o cliente (tensão entre manutenibilidade e agilidade, de um lado, e desempenho — CD-1 — de outro).
Risco	O desempenho de carregamento inicial do <i>frontend</i> não atinge a meta (LCP de 3,17 s, acima de 2,5 s, mesmo após <i>code-splitting</i>), confirmando quantitativamente o ponto de compromisso CD-1 (Seção 7.3.5).
Não risco	A imutabilidade do <code>user_type</code> e a validação de <code>provider_id</code> por <code>trigger</code> fornecem defesa em profundidade contra escalonamento e <i>spoofing</i> de prestador, reduzindo o risco de CS-2.

Fonte: elaborado pelos autores (2026).

6.5 Articulação com o plano de métricas

Cada cenário da árvore de utilidade vincula-se a uma ou mais métricas da Seção 5, de modo que a coleta quantitativa forneceu a evidência para confirmar ou refutar a análise qualitativa do ATAM. Essa articulação entre ATAM (qualitativo) e ISO/IEC 25010 (quantitativo) constitui o procedimento de avaliação proposto como contribuição do trabalho.

7 RESULTADOS

Esta seção consolida os resultados do trabalho. Os objetivos específicos 1 a 4 (delimitação, modelagem, documentação e definição de cenários) foram concluídos e são reportados em 7.1 e 7.2. Os objetivos 5 a 9 (execução de testes automatizados, análise estática, testes de segurança e de desempenho, e análise) foram **executados** em julho de 2026: as medições são apresentadas em 7.3, e a síntese por atributo de qualidade em 7.4.

Todos os valores quantitativos aqui reportados derivam de execuções registradas de forma reprodutível, no ambiente caracterizado na Seção 5.4, onde cada medição remete a uma ferramenta e versão, ao ambiente, à data e a um arquivo de evidência (conforme a Seção 5.1). Nenhum número é apresentado sem evidência correspondente. O registro completo está em <https://github.com/Richardy-Rodrigues/hubservi/blob/tcc-v2/docs/tcc/medicoes/registro-medicoes.md> e as saídas brutas de cada execução, em <https://github.com/Richardy-Rodrigues/hubservi/tree/tcc-v2/docs/tcc/medicoes/evidencias>; as capturas de tela dessas execuções compõem o Apêndice C (VIEIRA; COSTA, 2026c), ao qual cada subseção a seguir remete.

7.1 Definição e delimitação do estudo

- **Foco redefinido:** o trabalho foi reorientado da temática de recomendação para a **avaliação técnica da arquitetura de software**, mantendo o Hubservi como objeto de estudo. A recomendação permanece como módulo secundário (ordenação por popularidade e avaliação).
- **Objeto e instrumento (Seção 1.5):** o objeto de pesquisa é a avaliação; o Hubservi é o instrumento, desenvolvido pela equipe como pré-requisito metodológico — condição para o acesso irrestrito ao código, ao esquema e ao ambiente que a avaliação exige.
- **Modelo arquitetural consolidado: SPA + BaaS + Serverless**, corrigindo incoerência documental anterior que mencionava microsserviços (inexistentes no sistema real).
- **Atributos de qualidade selecionados (ISO/IEC 25010):** segurança, eficiência de desempenho, manutenibilidade (com ênfase em testabilidade) e confiabilidade.
- **Metodologia definida:** pesquisa aplicada, abordagem mista, objetivos exploratório-descritivos, estudo de caso com experimento técnico.

7.2 Modelagem e documentação da arquitetura

Foram produzidos e estão disponíveis no repositório do trabalho, em *docs/tcc/*, e reunidos no material suplementar (VIEIRA; COSTA, 2026a):

- descrição técnica completa da arquitetura, componentes, fluxos, persistência e regras de negócio (Seção 4);
- modelos UML — casos de uso, classes, sequência (autenticação, contratação e avaliação), componentes e implantação;

- modelo de processos de negócio (BPMN) para contratação e gerenciamento de *booking*;
- modelo de dados — DER e dicionário de dados, fiéis ao esquema real das *migrations*.

Esses artefatos atendem aos objetivos específicos 2 e 3 e fundamentam a árvore de utilidade e os cenários do ATAM (Seção 6).

7.3 Resultados das medições

A fase de execução configurou o ambiente reproduzível e coletou as métricas planejadas na Seção 5. O ambiente de teste de banco e de API foi levantado com uma instância **Supabase local** (contêiner Docker), contra a qual os cenários de autorização e de confiabilidade foram exercitados através da API real (PostgREST) — atendendo à exigência de teste de integração contra a API (Seção 5.2.1). Onde uma ferramenta nomeada no plano não estava disponível no ambiente, adotou-se um substituto da mesma classe, com a troca registrada: **k6 para autocannon** (teste de carga) e **Snyk para npm audit** (análise de composição de dependências).

7.3.1 Achado transversal de reprodutibilidade

Ao reconstruir o banco a partir apenas das *migrations* versionadas, constatou-se que **o histórico não reproduzia um sistema funcional do zero**: faltavam os *grants* de privilégios de API aos papéis *anon*, *authenticated* e *service_role* no *schema public*, provocando *permission denied* antes mesmo da avaliação de RLS. O sistema funciona em produção porque esses privilégios foram aplicados fora do versionamento. Corrigiu-se com uma *migration* que torna o histórico auto-suficiente, sem alterar políticas de segurança. É um resultado característico do paradigma BaaS — parte da configuração vive no serviço gerenciado e escapa ao controle de versão — e só se revela sob uma avaliação conduzida em ambiente limpo e reproduzível. A Figura 4 mostra os privilégios já concedidos após a correção (ver também Apêndice C, Figura C.8).

Figura 4 — Privilégios de API sobre *profiles* após a *migration* corretiva, verificados no banco reconstruído do zero

```
P-13 · F-01 - privilégios de API sobre public.profiles
execução de 2026-07-16 · commit ad89e6c
comando: psql: SELECT grantee, string_agg(privilege_type, ',') FROM information_schema.role_table_grants WHERE table_name = 'profiles'

fonte: docs/tcc/medicoes/evidencias/2026-07-16/grants-profiles-depois.txt

anon          | DELETE, INSERT, REFERENCES, SELECT, TRIGGER, TRUNCATE, UPDATE
authenticated | DELETE, INSERT, REFERENCES, SELECT, TRIGGER, TRUNCATE, UPDATE
service_role   | DELETE, INSERT, REFERENCES, SELECT, TRIGGER, TRUNCATE, UPDATE
```

Fonte: *psql* sobre o *stack* local; execução de 16 jul. 2026; saída bruta preservada em *evidencias/2026-07-16/grants-profiles-depois.txt*.

7.3.2 Segurança

Os cenários de autorização foram automatizados como testes de integração por papel. **Dois defeitos reais de autorização foram detectados, corrigidos e re-medidos**, seguindo o ciclo medir, detectar, corrigir e re-medir:

- **Exposição de PII a usuário autenticado:** a política de leitura de *profiles* (*USING (auth.uid() IS NOT NULL)*) permitia que qualquer usuário autenticado lesse *email* e *phone* de todos os perfis. Corrigido restringindo a leitura direta ao próprio registro; dados de contraparte seguem pela *view public_profiles*, sem PII.
- **Avaliação atribuída a prestador incorreto:** a política de inserção de *reviews* não validava que *provider_id* corresponde ao dono do serviço — assimetria em relação aos *bookings*, que já possuíam esse *trigger*. Corrigido com *trigger* de validação espelhado.

As Figuras 5 e 6 registram o par antes/depois desses dois defeitos. Na primeira, os testes falham e a saída exibe o vazamento literal — o campo *email* de outro usuário devolvido pela API a um cliente que não deveria vê-lo; na segunda, os mesmos testes passam, contra o mesmo cenário, após as *migrations* corretivas. Preservar os dois estados é o que distingue um defeito detectado de uma suíte que nunca reprovou nada (Seção 3.5).

Figura 5 — Antes da correção: falha dos testes de isolamento, com o vazamento de PII na saída

```
P-11 · F-02/F-03 - ANTES da correção (o instrumento reprova o sistema)
execução de 2026-07-16 · commit ad89e6c
comando: npm run test:integration (antes das migrations 20260716130000 e 20260716130100)

fonte: docs/tcc/medicoes/evidencias/2026-07-16/rls-furos-ANTES.log

RUN v3.2.7 C:/ServiceHub/hubservi

> tests/integration/rls-reviews.test.ts (3 tests | 1 failed) 619ms
✓ RLS - criacao de reviews > cliente com booking concluido cria review com provider correto (controle) 164ms
✓ RLS - criacao de reviews > cliente SEM booking concluido nao cria review (controle) 150ms
✗ RLS - criacao de reviews > cliente NAO deve avaliar um prestador que nao e o dono do servico 156ms
  → expected null not to be null
> tests/integration/rls-pii.test.ts (4 tests | 1 failed) 710ms
✓ RLS - exposicao de PII em profiles > anonimo nao le a tabela profiles diretamente (controle) 27ms
✗ RLS - exposicao de PII em profiles > autenticado NAO deve ler email/phone de outro usuario 134ms
  → expected { Object (email, phone) } to be null
✓ RLS - exposicao de PII em profiles > autenticado le o proprio perfil normalmente (controle) 126ms
✓ RLS - exposicao de PII em profiles > view public_profiles nao expoe email nem phone (controle) 287ms

Failed Tests 2

FAIL tests/integration/rls-pii.test.ts > RLS - exposicao de PII em profiles > autenticado NAO deve ler email/phone de outro usuario
AssertionError: expected { Object (email, phone) } to be null
- Expected:
null
+ Received:
{
  "email": "provider_b@test.local",
  "phone": "",
}

> tests/integration/rls-pii.test.ts:37:18
35| // Seguro: nenhum dado de outro perfil deve retornar. Enquanto o f_
36| // 'data' traz o email/phone de provider_b e esta asercao falha (e_
37| expect(data).toBeNull();
   |               ^
38| });
39|

[1/2]-

FAIL tests/integration/rls-reviews.test.ts > RLS - criacao de reviews > cliente NAO deve avaliar um prestador que nao e o dono do servico
AssertionError: expected null not to be null
> tests/integration/rls-reviews.test.ts:70:23
68| // Seguro: deve ser rejeitado. Enquanto o furo existir, o insert e_
69| // esta asercao falha (evidencia do furo).
70| expect(error).not.toBeNull();
   |               ^
71| });
72| });

[2/2]-

Test Files 2 failed (2)
Tests 2 failed | 5 passed (7)
Start at 11:28:31
Duration 2.06s (transform 76ms, setup 193ms, collect 53ms, tests 1.33s, environment 0ms, prepare 167ms)
```

Fonte: Vitest (integração); execução de 16 jul. 2026; saída bruta preservada em *evidencias/2026-07-16/rls-furos-ANTES.log*.

Figura 6 — Depois da correção: os mesmos testes aprovados, sem alteração no cenário

```
P-12 · F-02/F-03 - DEPOIS da correção (mesmos testes)
execução de 2026-07-16 · commit ad89e6c
comando: npm run test:integration (após as migrations de correção)

fonte: docs/tcc/medicoes/evidencias/2026-07-16/rls-furos-DEPOIS.log

RUN v3.2.7 C:/ServiceHub/hubservi

✓ tests/integration/rls-pii.test.ts (4 tests) 621ms
✓ tests/integration/rls-reviews.test.ts (3 tests) 560ms
✓ tests/integration/smoke.test.ts (4 tests) 414ms

Test Files 3 passed (3)
Tests 11 passed (11)
Start at 11:31:26
Duration 3.00s (transform 65ms, setup 197ms, collect 63ms, tests 1.60s, environment 0ms, prepare 268ms)
```

Fonte: Vitest (integração); execução de 16 jul. 2026; saída bruta preservada em *evidencias/2026-07-16/rls-furos-DEPOIS.log*.

Após as correções, a suíte de segurança (isolamento de perfis, serviços e *bookings*; bloqueio de escalonamento de *user_type*; regras de *review*) passa integralmente — **0 acessos indevidos nos cenários testados**, em 30 testes de integração distribuídos por nove arquivos. A análise de composição de dependências (*npm audit*) reportou 12 vulnerabilidades de severidade alta e 8 de severidade moderada, nenhuma crítica; destas, **apenas uma é de produção** (*react-router-dom*, *XSS via open redirect*, com correção disponível), sendo as demais ferramentas de *build* e de desenvolvimento, sem superfície de ataque em produção. O *supabase db lint* não acusou erros de *schema*. A varredura dinâmica (DAST/OWASP ZAP) permanece **pendente** de execução contra a URL de produção, por depender do ambiente de *hosting* real. As execuções correspondentes estão no Apêndice C (Figuras C.9, C.16 e C.17).

7.3.3 Confiabilidade

Verificados por testes de integração: a **máquina de estados do *booking rejeita transições inválidas** (0 transições inválidas aceitas); a *integridade referencial em exclusão de serviço não deixa registros órfãos (cascata)*; e o *fluxo crítico ponta a ponta (autenticação, serviço, booking* e avaliação)* completa com sucesso no caso válido. Atende aos critérios da Seção 5.2.5 (ver Apêndice C, Figura C.9).

7.3.4 Testabilidade

A suíte automatizada foi ampliada de 11 para **44 testes unitários e de componente**, além de 30 testes de integração. A **cobertura de linhas subiu de 18,0% para 32,0%** no agregado, concentrada nos módulos críticos, que passaram a **82% a 100%** (contexto de autenticação, formulários, camada de acesso a dados e *schemas* de validação). Definiu-se o limiar antes em aberto (Seção 5.2.3): piso global anti-regressão de 30% e piso de 75% nos módulos críticos, enumerados nominalmente e verificados automaticamente. Introduziu-se uma *factory* de *mock* compartilhada, reduzindo o esforço de escrever novos testes — evidência de que a testabilidade da arquitetura, atributo sob avaliação, melhorou.

Figura 7 — Cobertura de testes ao final da ampliação da suíte

```

P-03 · M-01b - cobertura de testes (Semana 5)
execução de 2026-07-16 · commit ad89e6c
comando: npm run test:coverage

fonte: docs/tcc/medicoes/evidencias/2026-07-16/coverage-semana5.txt
Cobertura Semana 5: lines 31.99% | branches 75% | funcs 48.38%

fonte: docs/tcc/medicoes/evidencias/2026-07-16/coverage-summary-semana5.json
campo extraído: .total
{
  "lines": {
    "total": 1747,
    "covered": 559,
    "skipped": 0,
    "pct": 31.99
  },
  "statements": {
    "total": 1747,
    "covered": 559,
    "skipped": 0,
    "pct": 31.99
  },
  "functions": {
    "total": 62,
    "covered": 30,
    "skipped": 0,
    "pct": 48.38
  },
  "branches": {
    "total": 144,
    "covered": 108,
    "skipped": 0,
    "pct": 75
  },
  "branchesTrue": {
    "total": 0,
    "covered": 0,
    "skipped": 0,
    "pct": 100
  }
}

LEITURA: linhas 31,99% (559/1747) · ramos 75% · funções 48,38%

```

Fonte: Vitest com [@vitest/coverage-v8](https://github.com/vitest/vitest); execução de 16 jul. 2026; saída bruta preservada em *evidencias/2026-07-16/coverage-semana5.txt*.

O estado inicial não dispõe de captura equivalente, e a assimetria é registrada em vez de contornada: a cobertura de 18,03% foi medida sobre uma árvore de trabalho anterior às correções e é, conforme o Apêndice B (VIEIRA; COSTA, 2026b), a única medição do trabalho que **não se reproduz em *commit* algum**. O que se preserva do estado inicial é a suíte do *baseline*, com seus 11 testes (Apêndice C, Figura C.2), não o percentual.

7.3.5 Eficiência de desempenho

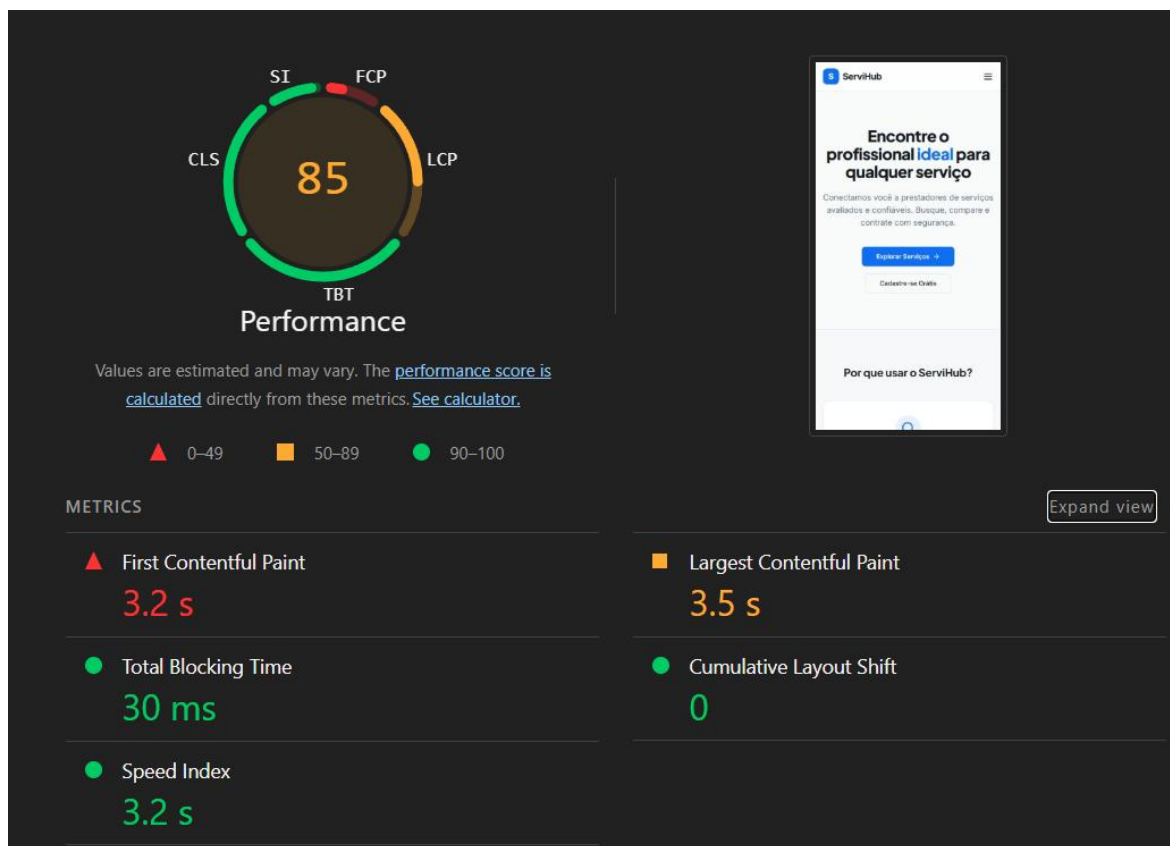
Distinguem-se duas camadas:

- **Backend sob carga (atende):** a listagem de serviços respondeu, sob 30 conexões concorrentes por 20 s, com **latência de cauda (p97,5) de 253 ms** (critério de 800 ms em p95, portanto atendido com folga) e **0% de erro** em 44.413 requisições, a uma vazão aproximada de 2.221 requisições por segundo.
- **Frontend e carregamento inicial (não atende):** o Lighthouse (mediana de três execuções, perfil móvel com limitação de CPU e de rede) registrou *performance score* de 85 e **LCP de 3,49 s** (critérios de 90 e de 2,5 s). Diagnosticou-se *bundle* único de 679

KB com rotas carregadas de forma *eager*; aplicou-se *code-splitting* por rota, reduzindo o *bundle* inicial em 28% (para 489 KB) e melhorando os índices (score de 88 e LCP de 3,17 s), que **permanecem abaixo da meta**. O TBT caiu de 11 ms para 0 ms e o CLS manteve-se em 0,000.

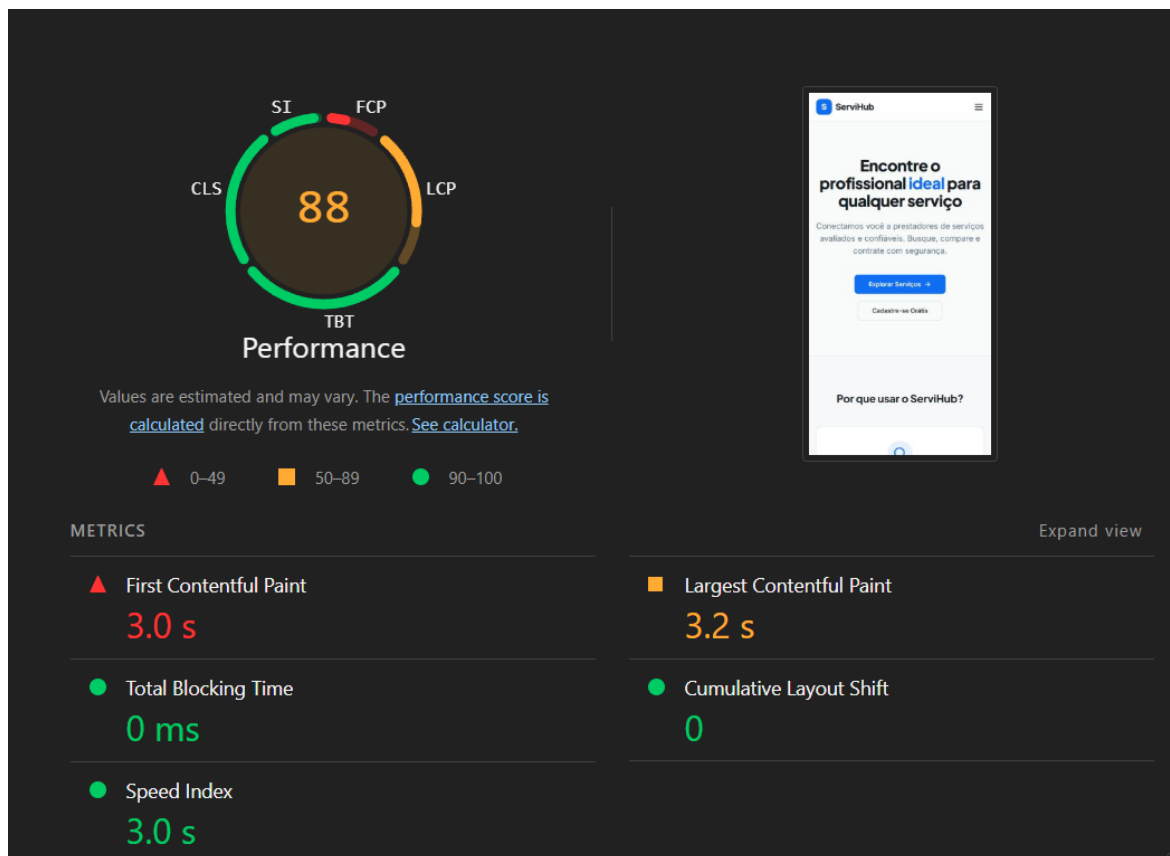
As Figuras 8 e 9 apresentam os relatórios do Lighthouse antes e depois do *code-splitting*, na execução mediana de cada rodada de três. A comparação torna visível tanto o ganho quanto o seu limite: os indicadores melhoram na direção esperada, e ainda assim o LCP permanece acima do critério de 2,5 s.

Figura 8 — Carregamento inicial antes do *code-splitting*: score de 85 e LCP de 3,49 s



Fonte: Lighthouse 12.8.2, perfil móvel; execução de 16 jul. 2026; saída bruta preservada em *evidencias/2026-07-16/lighthouse-3.json*.

Figura 9 — Carregamento inicial depois do *code-splitting*: score de 88 e LCP de 3,17 s



Fonte: Lighthouse 12.8.2, perfil móvel; execução de 16 jul. 2026; saída bruta preservada em *evidencias/2026-07-16/lighthouse-split-2.json*.

O contraste entre a API rápida e o carregamento lento localiza o gargalo de desempenho no *frontend* (peso do *bundle*), e não no *backend*. O ensaio de carga que sustenta o resultado de *backend* consta do Apêndice C, Figura C.15; note-se que ele corre sobre o volume de dados declarado na Seção 5.4, e vale como piso, não como projeção de operação real (Seção 5.5).

7.3.6 Manutenibilidade

A **modularização é sólida: 0 dependências circulares**, confirmado por duas ferramentas independentes (Madge, sobre 85 arquivos, e dependency-cruiser, sobre 93 módulos), com grafo de acoplamento saudável (núcleo estável e folhas voláteis). A **higiene de código está abaixo do ideal**: o *lint* não está zerado (19 violações na configuração atual e 25 sob configuração recomendada, incluindo funções com complexidade ciclomática elevada), e a **duplicação na camada de interface é de 4,55%** (acima da meta de 3%), com causa localizada — os dois painéis, de cliente e de prestador, compartilham 61 linhas quase idênticas. As execuções constam do Apêndice C (Figuras C.4, C.5, C.6 e C.7).

7.4 Síntese dos resultados

A avaliação técnica foi conduzida de ponta a ponta sobre um ambiente reproduzível e emite vereditos **por atributo e por camada**, em vez de um juízo único.

Tabela 15 — Síntese dos resultados por atributo de qualidade da ISO/IEC 25010

Atributo (ISO/IEC 25010)	Resultado
Segurança	Autorização declarativa (RLS e <i>triggers</i>) atende após a correção de dois defeitos reais; 1 CVE de produção a tratar; DAST pendente contra produção

Atributo (ISO/IEC 25010)	Resultado
Confiabilidade	Atende (máquina de estados, integridade referencial e fluxo crítico)
Testabilidade	Evoluiu (cobertura de 18% para 32%; módulos críticos entre 82% e 100%, com limiar protegido)
Eficiência de desempenho	<i>Backend</i> atende (p95 de 253 ms); <i>frontend</i> não atende (LCP de 3,17 s após otimização)
Manutenibilidade	Estrutura atende (0 ciclos); higiene de código abaixo do ideal (<i>lint</i> e duplicação nos painéis)

Fonte: elaborado pelos autores (2026).

O trabalho responde à questão de pesquisa demonstrando um **procedimento reprodutível de avaliação técnica** aplicável a arquiteturas BaaS/Serverless: modelagem fiel, cenários derivados dos atributos de qualidade, medição instrumentada com registro de evidências e o ciclo de detecção e correção de defeitos. O resultado não é um atestado de que o sistema é uniformemente bom — três defeitos reais foram encontrados e o desempenho de *frontend* não atinge a meta —, e é justamente essa capacidade de **localizar deficiências com precisão** que evidencia o valor do método. A reprodutibilidade é assegurada pelo registro em *docs/tcc/medicoes/* e pela integração contínua, que executa os *gates* automatizados a cada alteração.

8 CONCLUSÃO

Este trabalho investigou como avaliar tecnicamente uma arquitetura web baseada em BaaS e *Serverless* por meio de métricas e testes de Engenharia de Software. A resposta oferecida à questão de pesquisa é um **procedimento de avaliação arquitetural** que articula três elementos: o modelo de qualidade da ISO/IEC 25010 (2011), que fornece a taxonomia dos atributos; o método ATAM (CLEMENTS; KAZMAN; KLEIN, 2002), que organiza a análise qualitativa em cenários, riscos e pontos de compromisso; e um conjunto instrumentado de ferramentas de teste automatizado, análise estática, desempenho e segurança, que produz a evidência quantitativa. O procedimento foi aplicado integralmente à plataforma Hubservi, adotada como instrumento por permitir o acesso irrestrito ao código, ao esquema e ao ambiente que a avaliação exige.

Os resultados demonstram que o procedimento é capaz de **localizar deficiências com precisão**, e não apenas de emitir um juízo agregado. Três defeitos reais foram detectados, corrigidos e re-medidos: a exposição de dados pessoais a qualquer usuário autenticado, decorrente de política de RLS excessivamente permissiva; a ausência de validação do prestador na inserção de avaliações; e a incapacidade de o histórico de *migrations* reconstruir um sistema funcional do zero, por privilégios aplicados fora do controle de versão. Os três são achados característicos do paradigma BaaS — todos residem em configuração declarativa ou no serviço gerenciado, e nenhum se manifestava como erro funcional aparente na aplicação.

A avaliação também evidenciou que vereditos por camada são mais informativos do que um veredito único: enquanto o *backend* atende folgadoamente ao critério de desempenho, o carregamento inicial do *frontend* permanece abaixo da meta mesmo após otimização por *code-splitting*, o que confirma quantitativamente o ponto de compromisso identificado na análise ATAM — a arquitetura SPA transfere ao cliente parte do desempenho percebido. De modo análogo, a manutenibilidade apresenta estrutura sólida, sem dependências circulares, convivendo com higiene de código abaixo do ideal em *lint* e duplicação.

A principal contribuição é metodológica: um roteiro reprodutível, com registro de evidências e critérios fixados antes da coleta, transferível a outras aplicações que adotem o

mesmo paradigma. A contribuição prática, para a plataforma avaliada, materializa-se nas correções aplicadas e nos *gates* automatizados incorporados à integração contínua. Como **limitações**, registram-se a varredura dinâmica de segurança (DAST) ainda pendente de execução contra a URL de produção, a condução da avaliação pela própria equipe desenvolvedora — o que exigiu explicitar critérios previamente para mitigar viés — e a generalização restrita a um único caso. Como **trabalhos futuros**, propõem-se a execução do DAST em ambiente publicado, a aplicação do mesmo procedimento a outras plataformas BaaS para avaliar sua transferibilidade, a redução da duplicação identificada entre os painéis e a continuidade da otimização de carregamento do *frontend* até o alcance da meta de LCP.

REFERÊNCIAS

- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.
- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 3rd ed. Upper Saddle River: Addison-Wesley, 2012.
- CLEMENTS, Paul; KAZMAN, Rick; KLEIN, Mark. **Evaluating software architectures: methods and case studies**. Boston: Addison-Wesley, 2002.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. **ISO/IEC 25010**: systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. Geneva: ISO, 2011.
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.
- SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.
- VIEIRA, Pedro Conrado Fernandes; COSTA, Richardy Gabriel Rodrigues da. **Apêndice A — Diagramas (UML, BPMN e DER) e dicionário de dados**: material suplementar. Franca: Uni-FACEF, 2026a. Disponível em: <https://github.com/Richardy-Rodrigues/hubservi/blob/tcc-v2/docs/tcc/apendice-a-diagramas.md>. Acesso em: 20 ago. 2026.
- VIEIRA, Pedro Conrado Fernandes; COSTA, Richardy Gabriel Rodrigues da. **Apêndice B — Reprodução das medições**: material suplementar. Franca: Uni-FACEF, 2026b. Disponível em: <https://github.com/Richardy-Rodrigues/hubservi/blob/tcc-v2/docs/tcc/apendice-b-reproducao.md>. Acesso em: 20 ago. 2026.
- VIEIRA, Pedro Conrado Fernandes; COSTA, Richardy Gabriel Rodrigues da. **Apêndice C — Evidências de execução**: material suplementar. Franca: Uni-FACEF, 2026c. Disponível em: <https://github.com/Richardy-Rodrigues/hubservi/blob/tcc-v2/docs/tcc/apendice-c-evidencias.md>. Acesso em: 28 ago. 2026.